Lab 3: Application Requirements, HTTP and DNS

Network System Design | SE026 | 2026-27

Objective

Inspect HTTP and DNS separately, distinguish common failures, and turn user needs into requirements with observable acceptance tests.

1. Software and files

Use Python 3, a web browser with developer tools, and Command Prompt or a terminal. Wireshark and a network simulator are not required.

Download SE026-Lab-03-Resources-2026-27.zip. Right-click it and choose Extract All on Windows, then open the extracted SE026-Lab-03-Resources folder. Do not work inside the ZIP.

File	Purpose
README.md	Quick start and troubleshooting
web/index.html	Fictional page served by your local HTTP server
observations.csv	Blank table for actual test results
requirements.csv	Blank application-requirements worksheet
acceptance-tests.csv	Blank acceptance-test worksheet

2. Start the local server

On Windows, open the extracted folder in File Explorer. Click the address bar, type cmd, and press Enter. Check that dir web shows index.html.

Run py --version. If py is not found, try python --version and use python wherever the instructions use py. On macOS/Linux, use python3 in a terminal opened in the extracted folder.

Run: py -m http.server 8000 --bind 127.0.0.1 --directory web

Leave this terminal open. If Python is missing, install a stable Python 3 release from python.org. If port 8000 is occupied, use 8001 in the command and all local URLs.

3. Inspect a successful HTTP request

1. Open http://127.0.0.1:8000/index.html in the browser. Confirm the page title is Local networking lab. Type the URL; do not double-click the HTML file.

2. Open browser developer tools: F12 or Ctrl+Shift+I in Windows browsers. On macOS Chrome/Firefox, use Command+Option+I. Choose the Network tab.

3. Keep developer tools open. Enable Disable cache if available, clear the request list, then reload the page. If this option is unavailable, use a hard reload and inspect a fresh request.

4. Select index.html in the request list, not favicon.ico. Open Headers and record the URL, request method, status and response Content-Type. Record the HTTP version if your browser shows it; otherwise write not displayed.

5. Open Response or Preview to inspect the returned page content. Compare it with web/index.html in a text editor. Record actual results in observations.csv or your notes.

Read the evidence

The browser sends an HTTP request and the server returns a response. The method describes the requested operation; the status describes the response outcome; Content-Type describes the returned representation.

127.0.0.1 is the loopback address of your own computer. This activity tests a local service. It does not measure campus Internet performance and does not require public DNS.

Record	Your observation
URL / request method	_____
Status / Content-Type	_____
Response content / HTTP version	_____

Keep the HTTP server running for the next activity.

4. Compare three service outcomes

A. Missing resource

Open http://127.0.0.1:8000/missing.txt. Inspect that request in Network. Record its actual HTTP status and response. The file is deliberately absent: do not create it.

B. Stopped server

Return to the server terminal and press Ctrl+C. In the browser, request http://127.0.0.1:8000/index.html?test=stopped with cache disabled. Record the browser error and whether an HTTP response status exists. Error wording varies by browser.

C. Restore the service

Run the same server command again from the extracted resource folder. Request http://127.0.0.1:8000/index.html?test=restored and confirm the expected page returns. Record the result.

Explain the differences

For each outcome, state whether an HTTP response arrived, what remains working, and what failure is supported by your evidence. Use the command, URL and observed result so another student could repeat the test.

Case	Actual status or error	HTTP response received?
Working page	_____	_____
Missing resource	_____	_____
Stopped server	_____	_____
Restored service	_____	_____

If results look unchanged, check the URL and port, clear the browser request list, disable caching and make a fresh request. Do not change your firewall or network settings.

5. Separate DNS from application access

1. Open a second Command Prompt or terminal. Run: nslookup moodle.ccun.edu.kh

2. Record the resolver shown and the returned name/address information or exact error. The resolver address and the website address identify different roles. Multiple answers or changed addresses can be legitimate.

3. Open https://moodle.ccun.edu.kh/ in the browser while logged out. Record whether the public page loads or the error displayed. Do not enter a username or password for this activity.

4. Compare the lookup and browser results. Name resolution, transport connectivity, TLS and the web application are separate dependencies. A successful lookup alone does not prove the page or a file upload works.

5. Explain why the local 127.0.0.1 URL can work without a DNS lookup. If Internet access or nslookup is unavailable, retain the actual limitation and write the public-page test as a proposed test; do not invent a result.

DNS / application observation	Your result
Resolver / returned answer or error	_____
Public page outcome	_____
What each test establishes	_____

Reflection

1. How does a missing-resource HTTP response differ from a DNS failure?

2. Why can browser developer tools show HTTPS headers/content while an ordinary network capture observes encrypted application traffic?

3. Which activity deliberately changed only one service dependency, and how did you restore it?

6. Design requirements and acceptance tests

Use requirements.csv for Moodle access, video meetings and local printing. Specify the user, location, task, proposed target/condition and dependencies. Keep numeric targets as proposed assumptions pending stakeholder agreement.

A requirement states what users need; an acceptance test explains how to decide whether that requirement is met. Example format: From [location], [user group] can [task] under [conditions], meeting [observable target]. Fill the brackets with your own proposal.

For each application, consider the workload and number of simultaneous users, naming/service dependencies and permitted access. Do not claim a design target is an approved campus guarantee.

Use acceptance-tests.csv. Complete requirement, test location, action, expected result, evidence, and whether the test was observed or only proposed. A test should exercise the actual user task rather than rely on ping alone.

Application	What your plan must specify
Moodle	Define a file access task, test location, workload and evidence of correct content.
Video meeting	Define participant conditions and observable usability/quality criteria.
Local printing	Define an authorized user, printer/task and observable successful output.

The video and printing activities are design proposals: you do not need to configure a meeting service or connect to a real printer. Use fictional users and files. Include one failure scenario that affects a single dependency and explain which test would reveal it.

Finish

Save your observations, three requirements, three acceptance tests and reflection answers. CSV templates can be opened in a spreadsheet or replaced by equivalent tables in a document. Stop the temporary server with Ctrl+C.

Further help

Python HTTP server: https://docs.python.org/3/library/http.server.html